INTERNSHIP REPORT

ESP32-Based Intelligent Battery Management System

Embedded Systems Internship

Submitted to
ElevanceSkills Technology Private Limited

Submitted by: Parth Khairnar
Internship Role: Embedded Systems Intern
Internship Period: 12 September 2026 – 12 November 2026
Report Date: 13 October 2026

1. Introduction

This report summarizes the work carried out for the Embedded Systems Internship at ElevanceSkills. The assigned project is an ESP32-based Intelligent Battery Management System (BMS) prototype developed through simulation and IoT integration. The project focuses on monitoring simulated cell voltages, identifying voltage imbalance, managing protective states, presenting system information on an LCD, and sending selected telemetry to a Blynk dashboard. The report describes the project activities, skills practiced, challenges addressed, and evidence prepared for the final submission.

2. Background

A Battery Management System monitors battery-related parameters and helps detect conditions that may require protective action. In a multi-cell battery pack, differences between cell voltages can indicate imbalance, while abnormal readings or communication failures can affect system reliability. Embedded firmware can combine measurement processing, state-machine logic, local display, and remote telemetry to make system status easier to observe. For this educational prototype, potentiometers in the Wokwi simulation represent cell-voltage inputs; therefore, the results demonstrate firmware behaviour under simulated conditions rather than validating a physical battery pack.

3. Learning Objectives

- Understand how a modular embedded software design can separate measurement analysis, protection logic, display handling, and telemetry.
- Practice array-based cell monitoring and calculation of weakest/strongest cells and voltage imbalance.
- Understand non-blocking timing, hysteresis, debounce, and state-machine-based recovery.
- Explore LCD presentation techniques that reduce unnecessary redraws and improve readability.
- Integrate simulated embedded-system data with a Blynk IoT dashboard.
- Use GitHub to organize source code, project documentation, assumptions, bug records, and demonstration evidence.
- Develop a structured approach to testing, recording results, and preparing technical documentation.

4. Activities and Tasks

4.1 Modular Battery Management Engine

The project uses a cell-voltage array as the basis for monitoring multiple simulated inputs. The analysis approach identifies the minimum and maximum cell readings and calculates the voltage spread between them. This organization is intended to make the analysis reusable and easier to scale than separate, duplicated logic for every cell. The technical report documents the configured cell count, scaling considerations, and the implemented imbalance-trend behaviour.

4.2 Protection Relay and Safety Logic

Protection behaviour is organized around timed and state-based logic so that routine monitoring, display updates, and communication can continue while protection conditions are evaluated. Hysteresis and debounce concepts are used to reduce rapid switching around thresholds. Fault handling and recovery behaviour are documented with reference to the implemented simulation and its recorded evidence.

4.3 LCD Display Engine

The LCD provides a local view of cell measurements and system status. The display design aims to avoid unnecessary full-screen clearing and to present important information clearly. Status pages and fault-related display behaviour are documented in the technical report and supported by the available LCD screenshots.

4.4 Fault State Machine

The project defines four intended operating states: NORMAL, DEGRADED, FAILSAFE, and SHUTDOWN. These states provide a structured way to distinguish normal monitoring from degraded operation, protective response, and shutdown conditions. State transitions and recovery requirements are described in the technical report according to the implementation and evidence.

4.5 Event-Driven Telemetry and Blynk

The project integrates the ESP32 simulation with Blynk to present selected battery and system information remotely. The dashboard screenshots provide visual evidence of configured live values and dashboard/history views. Network-disconnection handling, offline queue behaviour, and ordered event replay should be described as implemented only where the final source code and test evidence confirm those behaviours.

4.6 Project Documentation and Repository

The GitHub repository was organized to provide a project overview, simulation link, source code, screenshots, assumptions document, and bug log. The README and screenshot guide help a reviewer navigate the project and locate supporting evidence. The repository and Wokwi simulation links are included below for review.

5. Skills and Competencies

- Embedded C/C++ firmware organization and working with ESP32-oriented simulation.
- Analog-input interpretation and conversion of simulated sensor values.
- Array-based processing and basic battery-cell imbalance analysis.
- State-machine concepts, non-blocking timing, and protection-system design principles.
- LCD interfacing and readable status presentation.
- IoT telemetry concepts and Blynk dashboard integration.
- Debugging through simulation, screenshots, and observed system behaviour.
- Technical writing, issue tracking, assumptions documentation, and GitHub repository organization.

6. Feedback and Evidence

The project evidence prepared for review includes the public GitHub repository, the Wokwi simulation, and screenshots representing normal operation, cell imbalance, failsafe detection, recovery, LCD output, Blynk dashboard information, and Wi-Fi indication. The assumptions document and bug log record design assumptions and known limitations. No formal mentor feedback is claimed in this report; the internship instructions state that mentor support is not available during the assignment.

GitHub repository: https://github.com/khairnarparth291-crypto/ESP32-Intelligent-Battery-Management-System

Wokwi simulation: https://wokwi.com/projects/477481844956329985

7. Challenges and Solutions

Interpreting simulated analog readings: Potentiometer values needed to be interpreted consistently as simulated cell voltages. The simulation was used to vary inputs and observe the resulting readings.

Maintaining stable protection behaviour: Hysteresis, debounce, and timed state transitions provide a structured approach to reducing relay chatter and controlling recovery.

Keeping LCD output readable: Avoiding unnecessary full-screen clearing and updating display fields selectively helps reduce flicker.

Organizing project evidence: A README, screenshots guide, assumptions document, and bug log were organized in the repository to make the work easier to review.

Documenting limitations honestly: The report distinguishes simulated input testing from real battery hardware validation and identifies areas that require further verification.

8. Outcomes and Impact

The project brings together several embedded-systems concepts in one simulation: cell monitoring, imbalance analysis, protective-state handling, local display, and IoT dashboard presentation. It also produced a structured GitHub repository containing the project overview and supporting documentation. The work provides a foundation for further testing and for a future hardware prototype. Since cell voltages are simulated, the current outcome should be treated as an educational software demonstration rather than a validated battery safety device.

9. Conclusion

The Embedded Systems Internship project provided practical experience in organizing ESP32-oriented firmware, analysing multiple simulated inputs, structuring protection behaviour, displaying system information, and presenting telemetry through Blynk. Preparing the source repository and supporting documents also strengthened documentation and verification practices. Before final submission, the report and its claims should be checked against the exact submitted firmware and test evidence. Any feature that has not been verified should be described as a design objective or limitation rather than a confirmed result.